
```
itinerary = [0,5,6,7,8,1,2,3,4,9]
```

We can do this type of lifting and moving with the following Python snippet, which will move a chosen section to a random location:

```
small = 1
big = 5
itinerary = [0,1,2,3,4,5,6,7,8,9]
tempitin = itinerary[small:big]
del(itinerary[small:big])
np.random.seed(random_seed + 1)
neighborids3 = math.floor(np.random.rand() * (len(itinerary)))
for j in range(0,len(tempitin)):
    itinerary.insert(neighborids3 + j,tempitin[j])
```

We can update our `perturb()` function so that it randomly alternates between these different perturbing methods. We'll do this by making another random selection of a number between 0 and 1. If this new random number lies in a certain range (say, 0–0.45), we'll perturb by reversing a subset of cities, but if it lies in another range (say, 0.45–0.55), we'll perturb by switching the places of two cities. If it lies in a final range (say, 0.55–1), we'll perturb by lifting and moving a subset of cities. In this way, our `perturb()` function can randomly alternate between each type of perturbing. We can put this random selection and these types of perturbing into our new function, now called `perturb_sa2()`, as shown in [Listing 6-5](#).

```
def perturb_sa2(cities,itinerary,time):
    neighborids1 = math.floor(np.random.rand() * (len(itinerary)))
    neighborids2 = math.floor(np.random.rand() * (len(itinerary)))

    itinerary2 = itinerary.copy()

    randomdraw2 = np.random.rand()
    small = min(neighborids1,neighborids2)
    big = max(neighborids1,neighborids2)
    if(randomdraw2 >= 0.55):
        itinerary2[small:big] = itinerary2[small:big][::-1]
    elif(randomdraw2 < 0.45):
        tempitin = itinerary[small:big]
        del(itinerary2[small:big])
        neighborids3 = math.floor(np.random.rand() * (len(itinerary)))
        for j in range(0,len(tempitin)):
```
